

Resolução — Exame Modelo 3 —

Paradigmas de Programação

Época de Recurso | Ano Letivo: 2025/2026

PARTE 1 – Perguntas Teóricas

Pergunta 1

Em Java, um array de objetos é uma estrutura de dados estática que armazena referências para objetos de um determinado tipo. A criação de um array de objetos envolve dois passos distintos que devem ser claramente compreendidos. O primeiro passo consiste na criação do próprio array, que aloca na Heap um bloco contíguo de memória com espaço para armazenar o número especificado de referências. Neste ponto, todas as posições do array são inicializadas com o valor null, pois ainda não existem objetos associados. O segundo passo consiste na criação individual de cada objeto que será armazenado nas posições do array, utilizando o operador new para cada instância.

Esta distinção é fundamental porque um erro frequente dos programadores iniciantes é assumir que ao criar um array de objetos, os objetos são automaticamente instanciados. Ao manipular arrays de objetos, é imprescindível verificar se a posição acedida contém uma referência nula antes de invocar qualquer método sobre essa referência, sob pena de ocorrer um NullPointerException em tempo de execução.

Os arrays em Java têm tamanho fixo porque são implementados como blocos contíguos de memória alocados na Heap, e a JVM não suporta o redimensionamento dinâmico destes blocos. Para simular o redimensionamento, é necessário criar um novo array com a dimensão pretendida e copiar os elementos do array original para o novo array.

```
public class ExemploArrays {  
    public static void main(String[] args) {  
        String[] nomes = new String[3];  
  
        nomes[0] = new String("Ana");  
        nomes[1] = new String("Bruno");  
  
        for (int i = 0; i < nomes.length; i++) {  
            if (nomes[i] != null) {  
                System.out.println(nomes[i]);  
            } else {  
                System.out.println("Posicao " + i + " esta vazia.");  
            }  
        }  
  
        String[] novosNomes = new String[5];  
        for (int i = 0; i < nomes.length; i++) {  
            novosNomes[i] = nomes[i];  
        }  
        novosNomes[2] = new String("Carlos");  
        novosNomes[3] = new String("Diana");  
    }  
}
```

Pergunta 2

O operador instanceof e o método getClass são ambos utilizados para verificação de tipos em Java, mas apresentam comportamentos distintos na presença de herança e interfaces.

O operador instanceof verifica se um objeto é uma instância de uma determinada classe ou de qualquer subclasse dessa classe, ou se implementa uma determinada interface. Avalia toda a cadeia de herança de forma ascendente, devolvendo true se o objeto for compatível com o tipo especificado em qualquer nível da hierarquia. É seguro relativamente a referências nulas, devolvendo false quando aplicado a null.

O método getClass devolve o objeto Class que representa a classe real e exata do objeto em tempo de execução, sem considerar a hierarquia de herança. A comparação com getClass é estrita: apenas devolve igualdade quando ambos os objetos pertencem exatamente à mesma classe, excluindo subclasses.

No contexto do método `equals`, a escolha entre ambas as abordagens tem implicações importantes. O uso de `instanceof` é mais adequado quando se pretende uma comparação flexível que funcione entre subclasses e implementações de uma interface, o que é particularmente útil quando a igualdade é definida num contrato de interface. O uso de `getClass` é mais adequado quando se pretende uma comparação estrita que garanta que ambos os objetos são exatamente do mesmo tipo concreto, preservando a simetria do contrato do `equals` sem efeitos colaterais na herança.

```
public class Animal {}  
public class Gato extends Animal {}  
  
public class Teste {  
    public static void main(String[] args) {  
        Gato gato = new Gato();  
  
        System.out.println(gato instanceof Gato);  
        System.out.println(gato instanceof Animal);  
        System.out.println(gato instanceof Object);  
  
        System.out.println(gato.getClass() == Gato.class);  
        System.out.println(gato.getClass() == Animal.class);  
    }  
}
```

Pergunta 3

Os membros estáticos em Java são elementos de uma classe que pertencem à própria classe e não a nenhuma instância individual. São declarados com a palavra reservada `static` e existem independentemente da criação de objetos da classe.

Os atributos estáticos são variáveis partilhadas por todas as instâncias da classe. Existe uma única cópia de cada atributo estático na memória, que é alocada quando a classe é carregada pela JVM e permanece durante toda a execução do programa. Todas as instâncias da classe acedem e modificam o mesmo atributo estático. Os atributos de instância, por contraste, existem individualmente em cada objeto, ocupando espaço próprio na Heap de cada instância.

Os métodos estáticos são métodos que podem ser invocados diretamente através do nome da classe, sem necessidade de criar uma instância. Um método estático não pode aceder diretamente a atributos ou métodos de instância, porque não existe nenhum objeto de contexto

associado à sua invocação. Também não pode utilizar a referência `this`, pois esta aponta para a instância corrente, que não existe num contexto estático.

Um caso de uso clássico para atributos estáticos é o padrão de contagem de instâncias, onde um atributo estático é incrementado em cada invocação do construtor para manter o registo do número total de objetos criados.

```
public class Contentor {  
    private static int totalContentores = 0;  
    private String codigo;  
  
    public Contentor(String codigo) {  
        this.codigo = codigo;  
        totalContentores++;  
    }  
  
    public static int getTotalContentores() {  
        return totalContentores;  
    }  
  
    public String getCodigo() {  
        return codigo;  
    }  
  
    public static void main(String[] args) {  
        Contentor c1 = new Contentor("C-001");  
        Contentor c2 = new Contentor("C-002");  
        Contentor c3 = new Contentor("C-003");  
        System.out.println("Total de contentores: " + Contentor.getTotalContentores());  
    }  
}
```

Pergunta 4

As classes abstratas e as interfaces em Java apresentam distinções fundamentais na forma como gerem a herança, o estado interno e os construtores. Relativamente à herança, uma classe abstrata suporta apenas herança simples, o que significa que uma classe derivada pode estender no máximo uma única classe base. Em contrapartida, as interfaces suportam herança múltipla de tipo, permitindo que uma classe implemente várias interfaces em simultâneo.

No que diz respeito ao estado, as classes abstratas podem conter atributos de instância com qualquer modificador de acesso, permitindo armazenar dados mutáveis e partilhá-los com as subclasses. As interfaces não podem possuir variáveis de instância, definindo apenas constantes que são implicitamente públicas, estáticas e finais. Adicionalmente, as classes abstratas podem definir construtores que são invocados pelas subclasses por intermédio da instrução `super` para inicializar o estado herdado. As interfaces não contêm construtores porque não podem ser instanciadas.

É adequado optar por uma classe abstrata quando se pretende partilhar estado e código comum entre classes estreitamente relacionadas na mesma hierarquia. É preferível utilizar uma interface quando se deseja definir um contrato de comportamento comum que pode ser implementado por classes dispersas em diferentes hierarquias e sem ligação direta.

```
public abstract class Transporte {
    private String identificador;

    public Transporte(String identificador) {
        this.identificador = identificador;
    }

    public String getIdentificador() {
        return identificador;
    }

    public abstract double calcularCusto(double distancia);
}

public interface Rastreavel {
    String getLocalizacao();
}

public class CamiaoRecolha extends Transporte implements Rastreavel {
    private double custoPorKm;
    private String localizacao;

    public CamiaoRecolha(String identificador, double custoPorKm) {
        super(identificador);
        this.custoPorKm = custoPorKm;
        this.localizacao = "Base";
    }

    @Override
    public double calcularCusto(double distancia) {
        return distancia * custoPorKm;
    }

    @Override
    public String getLocalizacao() {
        return localizacao;
    }
}
```

PARTE 2 – Programação em Java

Pergunta 1a

```
public class ContainerImpl implements Container {  
    private static final int MAX_MEASUREMENTS = 50;  
    private String code;  
    private ItemType type;  
    private double capacity;  
    private Measurement[] measurements;  
    private int numberOfMeasurements;  
  
    public ContainerImpl(String code, ItemType type, double capacity) {  
        if (code == null) {  
            throw new IllegalArgumentException("O código não pode ser nulo.");  
        }  
        if (type == null) {  
            throw new IllegalArgumentException("O tipo não pode ser nulo.");  
        }  
        if (capacity <= 0) {  
            throw new IllegalArgumentException("A capacidade deve ser positiva.");  
        }  
        this.code = code;  
        this.type = type;  
        this.capacity = capacity;  
        this.measurements = new Measurement[MAX_MEASUREMENTS];  
        this.numberOfMeasurements = 0;  
    }  
  
    @Override  
    public String getCode() {  
        return this.code;  
    }  
  
    @Override  
    public ItemType getType() {  
        return this.type;  
    }  
  
    @Override
```

```
public double getCapacity() {
    return this.capacity;
}

@Override
public Measurement[] getMeasurements() {
    Measurement[] result = new Measurement[numberOfMeasurements];
    for (int i = 0; i < numberOfMeasurements; i++) {
        result[i] = measurements[i];
    }
    return result;
}

@Override
public Measurement getLastMeasurement() {
    if (numberOfMeasurements == 0) {
        return null;
    }
    return measurements[numberOfMeasurements - 1];
}

@Override
public void addMeasurement(Measurement measurement) throws ContainerException {
    if (measurement == null) {
        throw new ContainerException("A medicao nao pode ser nula.");
    }
    if (measurement.getValue() > this.capacity) {
        throw new ContainerException("O valor da medicao excede a capacidade do contentor.");
    }
    if (measurement.getValue() < 0) {
        throw new ContainerException("O valor da medicao nao pode ser negativo.");
    }
    if (numberOfMeasurements >= MAX_MEASUREMENTS) {
        for (int i = 0; i < numberOfMeasurements - 1; i++) {
            measurements[i] = measurements[i + 1];
        }
        numberOfMeasurements--;
    }
    measurements[numberOfMeasurements] = measurement;
    numberOfMeasurements++;
}
```



```
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (obj == null) {
        return false;
    }
    if (!(obj instanceof Container)) {
        return false;
    }
    Container other = (Container) obj;
    return this.code.equals(other.getCode());
}

@Override
public String toString() {
    return "Contentor [Codigo: " + code + " | Tipo: " + type + " | Capacidade: " + capacity + "kg";
}
}
```

Pergunta 1b

```
public class TestContainer {  
    public static void main(String[] args) {  
        ContainerImpl c1 = new ContainerImpl("C-001", ItemType.PERISHABLE_FOOD, 100.0);  
        ContainerImpl c2 = new ContainerImpl("C-001", ItemType.MEDICINE, 200.0);  
        ContainerImpl c3 = new ContainerImpl("C-002", ItemType.CLOTHING, 150.0);  
  
        System.out.println("Codigo c1: " + c1.getCode());  
        System.out.println("Tipo c1: " + c1.getType());  
        System.out.println("Capacidade c1: " + c1.getCapacity());  
  
        System.out.println("Ultima medicao (sem medicoes): " + c1.getLastMeasurement());  
        System.out.println("Numero de medicoes inicial: " + c1.getMeasurements().length);  
  
        try {  
            MeasurementImpl m1 = new MeasurementImpl(45.0, "2026-07-01");  
            c1.addMeasurement(m1);  
            System.out.println("Medicao adicionada com sucesso.");  
            System.out.println("Ultima medicao: " + c1.getLastMeasurement().getValue());  
        } catch (ContainerException e) {  
            System.out.println("Erro: " + e.getMessage());  
        }  
  
        try {  
            MeasurementImpl mInvalida = new MeasurementImpl(200.0, "2026-07-02");  
            c1.addMeasurement(mInvalida);  
        } catch (ContainerException e) {  
            System.out.println("Erro esperado (excede capacidade): " + e.getMessage());  
        }  
  
        try {  
            c1.addMeasurement(null);  
        } catch (ContainerException e) {  
            System.out.println("Erro esperado (nulo): " + e.getMessage());  
        }  
  
        System.out.println("Igualdade c1 com c2 (mesmo codigo): " + c1.equals(c2));  
        System.out.println("Igualdade c1 com c3 (codigos diferentes): " + c1.equals(c3));  
        System.out.println("Igualdade c1 com nulo: " + c1.equals(null));  
    }  
}
```

```
        System.out.println("Representacao textual: " + c1.toString());  
    }  
}
```

Pergunta 2a

```
public class PickingMapImpl implements PickingMap {

    private boolean hasContainerOfType(AidBox aidbox, ItemType type) {

        if (aidbox == null || type == null) {
            return false;
        }
        Container[] containers = aidbox.getContainers();
        if (containers == null) {
            return false;
        }
        for (int i = 0; i < containers.length; i++) {
            if (containers[i] != null && containers[i].getType() == type) {
                return true;
            }
        }
        return false;
    }

    private boolean needsCollection(Container container) {
        if (container == null) {
            return false;
        }
        Measurement last = container.getLastMeasurement();
        if (last == null) {
            return false;
        }
        if (container.getType() == ItemType.PERISHABLE_FOOD) {
            return true;
        }
        return last.getValue() > (container.getCapacity() * 0.7);
    }

    @Override
    public AidBox[] getPickingMap(IIInstitution inst, ItemType type) {
        return new AidBox[0];
    }
}
```

Pergunta 2b

```
public class PickingMapImpl implements PickingMap {

    private boolean hasContainerOfType(AidBox aidbox, ItemType type) {

        if (aidbox == null || type == null) {
            return false;
        }
        Container[] containers = aidbox.getContainers();
        if (containers == null) {
            return false;
        }
        for (int i = 0; i < containers.length; i++) {
            if (containers[i] != null && containers[i].getType() == type) {
                return true;
            }
        }
        return false;
    }

    private boolean needsCollection(Container container) {
        if (container == null) {
            return false;
        }
        Measurement last = container.getLastMeasurement();
        if (last == null) {
            return false;
        }
        if (container.getType() == ItemType.PERISHABLE_FOOD) {
            return true;
        }
        return last.getValue() > (container.getCapacity() * 0.7);
    }

    @Override
    public AidBox[] getPickingMap(IIInstitution inst, ItemType type) {
        if (inst == null || type == null) {
            return new AidBox[0];
        }
        AidBox[] aidBoxes = inst.getAidBoxes();
        if (aidBoxes == null || aidBoxes.length == 0) {
```

```
        return new AidBox[0];
    }
    AidBox[] tempResult = new AidBox[aidBoxes.length];
    int count = 0;
    for (int i = 0; i < aidBoxes.length; i++) {
        if (aidBoxes[i] == null) {
            continue;
        }
        if (!hasContainerOfType(aidBoxes[i], type)) {
            continue;
        }
        Container[] containers = aidBoxes[i].getContainers();
        if (containers == null) {
            continue;
        }
        boolean precisaRecolha = false;
        for (int j = 0; j < containers.length; j++) {
            if (containers[j] == null) {
                continue;
            }
            if (containers[j].getType() == type && needsCollection(containers[j])) {
                precisaRecolha = true;
                break;
            }
        }
        if (precisaRecolha) {
            tempResult[count] = aidBoxes[i];
            count++;
        }
    }
    AidBox[] finalResult = new AidBox[count];
    for (int i = 0; i < count; i++) {
        finalResult[i] = tempResult[i];
    }
    return finalResult;
}
}
```